CSC_51053_EP, Exercise Session: Recovery in RDBMSs

Ioana Manolescu, Madhulika Mohanty, Garima Gaur

November 28, 2025

1 ACID concept review

1. If we assume disks never crash, do we still need to make an effort to enforce durability? Why?

   Yes, because all changes to data pages are made while the data pages are in memory, and main memory is volatile. Moreover, when using no-force (pages are not written to disk immediately upon transaction commit – "immediately" does not exist, anyway, when one thinks of the time it takes to write a potentially large set of pages), the changes to these in-memory pages could be completely lost (if , for instance, power loss results in main memory information disappearing). This is why we still need to ensure durability, e.g., using logs and the ARIES protocol.
   In reality, of course, disks do crash; not having a replica of the database is equivalent to assuming the disk never crashes! A reformulated question could then be: "if we assume that we have random access to stable storage at a speed comparable to that of RAM, do we need to enforce durability?", or in other words "if RAM is never lost, do we need to enforce durability?"[1]. The answer would probably be that in this case durability is less relevant. However, the current perspective is that NVRAM storage may replace RAM but not disk, as its storage capacity is not large enough. Therefore large databases still would have to take measures to ensure disk writes are durable.

2. What data structures which are part of the Databases Management System (DBMS) are lost when the server crashes?

   The short answer is "everything stored in memory", because memory is allocated to a *process*, e.g., the database server, and de-allocated when the process is finished (this means that another process can overwrite it). In details, a crash leads to losing:

   - The memory buffer, containing all the buffer pages, together with the information about the number of users of a page, the usage statistics of every page that allow to determine which page should be the next one evicted from the buffer etc.
     Note that each in-memory page contains a lastLSN (Log Serial Number of the last log record that attempted to modify this page) value. This may or may not be the one present in the disk version of the page.
   - The Transaction Table of the recovery manager.
   - The Dirty Pages table; note that this one refers to the pages from the memory buffer but is a separate data structure.
   - All the lock-related data structures: which transaction had which locks, which transactions are in queue for which lock etc.
   - The tail of the log (the part that is still in memory, not yet flushed to stable storage).

3. As a way to ensure durability, how is logging different from replication?

   First, the log stores information that is *much less voluminous* than the data. For instance, for an update of 4 bytes in a page, the corresponding log record stores just the before and after images of the 4 bytes; a replica of the database needs to store the whole page.
   Second, the log stores information related to the *actions* on the data (one log record per update). If a page is written N times, there will be N corresponding log records. A replica only stores another *copy of the data* and has no information about the actions that have lead to the last stored version of the data.

[1] This is not so unrealistic with NVRAM (non-volatile RAM), "the next big thing" in terms of fast yet durable storage. Several NVRAM solutions are under development and a few are currently on the market, e.g., 3D XPoint by Intel and Micron sold under the brand name Optane (Intel) and QuantX (Micron).

4. What is the utility of checkpointing? What are the advantages and disadvantages of having frequent checkpoints (vs. having them more rarely)? The screen capture below can be used as a hint.

```
[223-235:~/pgData] ioanamanolescu% pg_ctl start -D ~/pgData/
server starting
[223-235:~/pgData] ioanamanolescu% LOG:  database system was shut down at 2019-11-15 09:02:09 CET
LOG:  MultiXact member wraparound protections are now enabled
LOG:  database system is ready to accept connections
LOG:  autovacuum launcher started
LOG:  checkpoints are occurring too frequently (8 seconds apart)
HINT:  Consider increasing the configuration parameter "max_wal_size".
LOG:  checkpoints are occurring too frequently (4 seconds apart)
HINT:  Consider increasing the configuration parameter "max_wal_size".
LOG:  checkpoints are occurring too frequently (2 seconds apart)
HINT:  Consider increasing the configuration parameter "max_wal_size".
LOG:  checkpoints are occurring too frequently (3 seconds apart)
```

Checkpointing writes to the log (thus, eventually, on stable storage) the state of the Transaction Table and of the Dirty Pages Table. These are memory-resident data structures, thus, without checkpointing, they would be completely lost after a crash.
During ARIES recovery, Analysis examines the stable-storage log since the last checkpoint. Analysis goes through all these records to determine which are the loser transactions (that need to be undone).

- If the last checkpoint is in a distant past (rare checkpointing), Analysis will have to examine a long history, during which possibly many transactions have started and committed and their changes may have even gone to disk; thus, Analysis and possibly Redo will take longer.
- If the last checkpoint is recent (frequent checkpointing), recovery is faster, however, checkpointing has its own cost: it adds potentially large records to the log, records which have to be flushed to obey the WAL protocol as transactions write their changes to disk and/or commit. Therefore, frequent checkpointing slows down the execution of transactions.

It is also useful to know that unlike the textbook presentation of the ARIES algorithm, *some systems write the full dirty pages themselves in the checkpoint record*, or give an option to control whether the full page is written, as documented for instance at `https://www.postgresql.org/docs/current/wal-configuration.html`. Even in Postgres, by default the full page is written in the checkpoint log.

5. What tools and algorithms seen in the course enforce:

   (a) Atomicity?

   Atomicity is ensured though the recovery module, specifically by all the steps that undo the changes of transactions whose commit record did not make it to the stable-storage part of the log before transaction abort or system crash.

   (b) Consistency?

   The DBMS allow users to specify application constraints through the DDL (data declaration language), when creating tables or when altering them.
   The DBMS enforces consistency by aborting transactions that attempt to violate a constraint.

   (c) Isolation?

   Isolation is ensured as soon as the schedule of execution of a set of transactions is serializable, that is, equivalent to one sequential execution of the transactions. In turn, serializable schedules are ensured for instance by concurrency control mechanisms, such as pessimistic ones (lock-based) or optimistic ones.

   (d) Durability?

   First, durability is ensured by the Redo phase of the crash recovery algorithm; it makes sure that the writes of transactions which had written their Commit record to the log before the crash, are actually propagated to disk.
   Second, durability is also ensured by the scheduling mechanism (e.g., through locks) that only results in recoverable schedules. This allows never to undo a committed transaction.
   Other mechanisms which enforce durability are: storing the log in a different machine than the one with the disk where the data is stored; replicating the log and/or the data, again preferrably storing a replica in a remote place to reduce the risk of having it destroyed at the same time as the primary copy.

6. How does concurrency control impact the *performance* of query (and update) processing? Is the impact positive or negative? Compare:

   - A setting where execution is alway serial (thus, no concurrency, and no concurrency control) with a concurrent setting with concurrency control in place.
   - A normal, concurrent setting with concurrency control, using *"less" concurrency control* vs. *"more" concurrency control*.

   Concurrency control has the *goal* of supporting concurrent execution of queries and updates (simply viewed as transactions) correctly, that is, preserving Isolation and Consistency.

   - Compared with a serial execution with no concurrency: concurrency does not improve the execution time for the first transaction in the serial schedule, however, it is likely to shorten the execution of the others, to the extent that they need different resources and thus do not wait too much on each other. Note that it is *concurrency* who improves the performance, not *concurrency control* (which constrains concurrency, in order to preserve I and C).
   - Compared with a concurrent execution without concurrency control (or with less control), concurrency control strictly slows down things, because of its own overhead (lock management, wasted and repeated work when aborting and restarting transactions etc.)

   Thus: among ACID executions, *concurrency* helps improve performance, while *concurrency control* has a (usually modest) cost; if we compare also to non-ACID concurrent execution, concurrency control only brings a performance overhead. This is the price to pay for I and C.

7. What are the main aspects of recovery with an impact on performance of query (and update) processing? Is the impact positive or negative?

   The recovery mechanisms seen in the course have the *goal* of supporting Atomicity and Durability for no-force, steal transaction execution, which is more efficient than the alternatives.
   The features implementing recovery (logging with all its consequences, maintaining the transaction table, the log manager, flushing log entries as part of transaction execution, managing a pageLSN in each page etc.) all have a negative impact on performance. This is the price to pay for A and D.
   Several parameters which allow to tune the recovery mechanisms and thus also impact recovery costs; setting them requires a good understanding of the system, see, e.g., the respective documentation of the RDBMS, or textbooks such as [2] (caveat: in a technology domain, books deprecate! In general be sure to rely on up-to-date technical material).

2 Recovery

1. What are the situations that trigger log entries being flushed to persistent storage?

   (a) According to the WAL protocol:
      i. Before the buffer manager forces a page to disk, the log entries prior to and up to the last update of this page are flushed to disk (WAL1).
      ii. When a transaction commits, it first adds a Commit record to the log, then the log is flushed up to and including the last operation of this transaction, then an End record is written to the log (WAL2).

   (b) Checkpointing flushes "begin checkpoint" and "end checkpoint" entries to disk".

   (c) Log entries may be flushed to disk simply when the memory space occupied by the log becomes full, or at a regular time interval.

2. Why does REDO go over log records in the forward order, while UNDO goes over the log records in the reverse order?

   REDO goes in the forward order to make sure that in case of repeated (successive) writes that need to be redone over the data by a transaction that has committed (thus, must be made durable), the data will be brought to the state corresponding to the last write.

UNDO goes in the backward order to make sure that in case of successive writes to the same data item by one or several transactions that need to be undone, the original value is the one that will be retained at the end of UNDO.

3. [1] Consider the schedule in Figure 1. Following the crash, state precisely what is done during: Analysis; Redo; Undo.

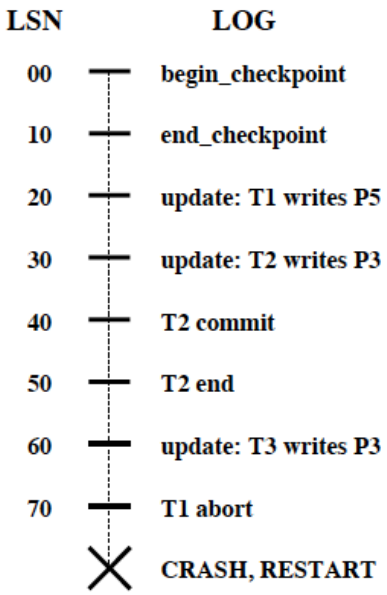


Figure 1: Sample schedule for recovery including a crash.

Analysis starts at the last checkpoint, thus at LSN 00. In this schedule, we have no information on what happened before that checkpoint, thus we assume that the TT and DPT we find are empty. Then:

(a) Log record with LSN 20: Analysis adds T1 to the TT (status: U, to be undone), P5 to the DPT, and sets the lastLSN of T1 to 20 and the recLSN of P5 to 20.

(b) LSN 30: Analysis adds T2 to the TT (status: U), P3 to the DPT, and sets the lastLSN of T2 to 30 and the recLSN of P3 to 30.

(c) LSN 40: Analysis sets the status of T2 to C (committed).

(d) LSN 50: Analysis removes T2 from TT.

(e) LSN 60: Analysis adds T3 to the TT (status: U) and sets its lastLSN to 60. P3 is already in the DPT and its recLSN does not change (the first log record that made it dirty is still 30).

(f) LSN 70: no impact (any transaction not found ended or committed is to be aborted/undone, anyway!)

At the end of the analysis, the transactions to be undone are T1 (LSN 20) and T3 (LSN 60). The transaction T2 (LSN: 40) is to be committed. The dirty pages are P5 (recLSN: 20) and P3 (recLSN: 30). **Redo** starts at the smallest LSN in the DPT built by Analysis, thus at 20.

(a) LSN 20: the update affects P5 which is dirty. The recLSN of P5 is 20. Thus, we fetch P5 to get its pageLSN, assumed before the beginning of history. Therefore, we redo the update of P5: P5 is brought in memory; the update described in the record at LSN 20 is re-applied; the pageLSN of P5 is set (in memory) to 20.

(b) LSN 30: the update affects P3 which is dirty. The recLSN of P3 is 30, thus we fetch it to get its pageLSN, and similarly redo on P3 the update described by the record at LSN 30. The pageLSN of P3 is set to 30.

(c) LSN 40: nothing.

(d) LSN 50: nothing.

(e) LSN 60: P3 is dirty; its recLSN is 30, and its pageLSN is 30. We redo the write described at LSN 60.

At the end of Redo, P5 reflects the write of T1, while P3 reflects the last write, that of T3. The loser transactions are T1 (lastLSN: 20) and T3 (lastLSN: 60).
Undo proceeds as follows:

(a) largestLSN is 60, the update of T3 (LSN 60, on P3). Write a CLR at LSN 80 stating that the update will be undone; the record at LSN 60 doesn't have a prevLSN, thus the CLR has undonextLSN=null. Undo the update: P3 reverts to its state before 60 (thus, to the committed write of T2). No addition to the toUndo list.
In this example, at this point, T3 has been undone.

(b) largestLSN is 20, the update of T1 (LSN 20, on P1). Write a CLR at LSN 90 to state the undoing; this CLR has undonextLSN=null. Undo the update: P5 reverts to the state before this schedule. No addition to the toUndo list.
In this example, at this point, T1 has been undone.

4. [1] Consider the execution in Figure 2.

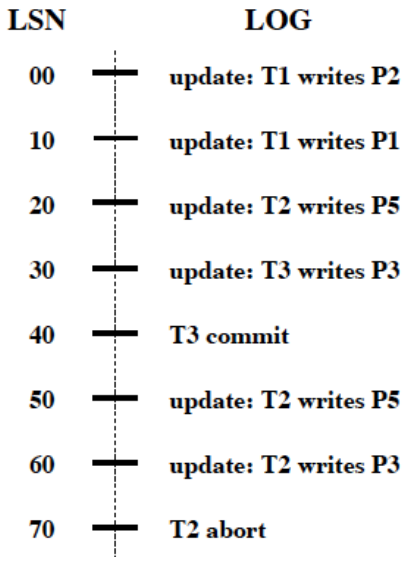


Figure 2: Sample schedule for recovery including an abort of T2.

(a) Extend the figure to show prevLSN and undonextLSN values.
See Figure 3.

(b) Describe the actions taken to rollback transaction T2.
Once the transition table has been computed and the redo phase has run, the first record that is rolled back is $T2$. It just happens that here there are no other records to undo until we have completely undone $T2$ so we can safely focus on $T2$. For each action starting from LSN 60, then LSN50, then LSN 20, then we first insert the corresponding CLR as depicted in the Figure, then undo the action.

(c) Show the log after T2 is rolled back, including all prevLSN and undonextLSN values in log records. See Figure 3 for the CLR logs.

5. Examine the schedule given below. There are three transactions: T_1, T_2, T_3. Initially, the salary is equal to 1 and the tax is equal to 2. Assume WAL and ARIES are used.

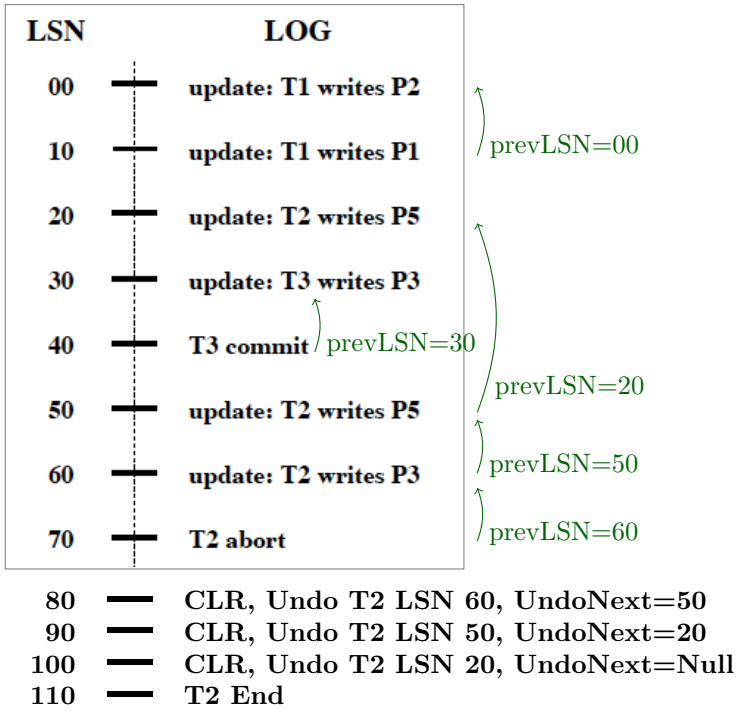


Figure 3: Extended schedule.

	T_1	T_2	T_3
0			START
1			READ tax
2			tax := tax +1
3	START		
4	READ salary		
5	salary := salary +1		
6			WRITE tax
7			COMMIT
8		START	
9		READ tax	
10		READ salary	
11		tax := tax +1	
12		WRITE tax	
13		COMMIT	
14	READ tax		
15	tax := tax + salary		
16	WRITE tax, salary		
17	**System Crashes**		

(a) Show the log entries that would be generated by this execution. Use the same notation used in class. For each log entry, indicate what line above generates it.

Only the updates need to be logged. Thus, assuming tax is in P1 and salary is on P2, we will have the following entries:
Update: T3 writes P1
T3 commit
Update: T2 writes P1
T2 commit
Update: T1 writes P1
Update: T1 writes P2

(b) When the system is rebooted and tries to recover from the crash which transaction(s) should be undone and which may be redone ?

Assuming that the last checkpoint happened before these transactions started, T2 and T3 would be redone and T1 would be undone as it is the only transaction active during the crash and hence, will be identified as a "loser" transaction by ARIES in the recovery phase.

References

[1] Raghu Ramakrishnan and Johannes Gehrke. *Database management systems (3rd ed.)*. McGraw-Hill, 2003.

[2] Dennis Shasha and Philippe Bonnet. *Database Tuning: Principles, Experiments, and Troubleshooting Techniques*. Morgan Kaufmann, 2002.